

事务的基本概念

(2026 春季)



1. 事务
2. 定义事务
3. 有关事务的SQL语句
4. 事务的ACID特性
5. 事务活动

1、事务 (1)

- ❑ **事务(Transaction)**是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位。

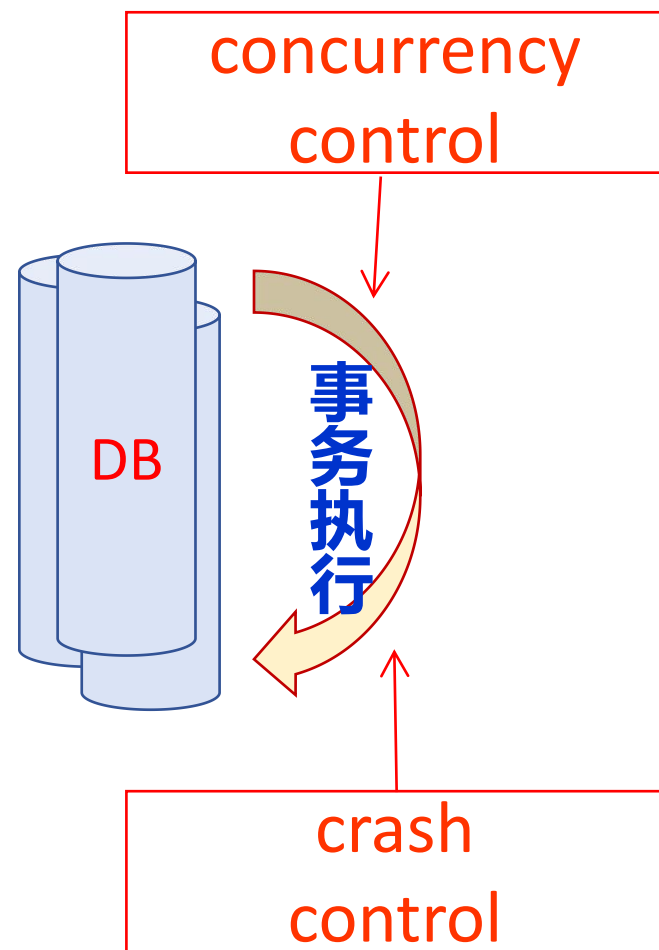
- ❑ **事务和程序**是两个概念
 - 在关系数据库中，一个事务可以是一条**SQL**语句、一组**SQL**语句或整个程序
 - 一个程序通常包含多个事务

- ❑ **事务**是恢复和并发控制的基本单位

1、事务 (2)

□ 为什么要引入 ‘事务’ 概念？

- 为了确保实现数据库的数据完整性 (integrity of the database)
- 在一个数据库系统中，如果所有数据都满足数据完整性约束的定义，那么就称数据库处于一个一致性状态(consistent state)
- 影响数据完整性的三个因素
 - concurrency
 - abort
 - crash
- 在数据库管理系统中引入事务处理功能，可以确保在任何情况下，都能保证数据库中数据的正确性。



事务的例子

- ❑ [例]银行的转账业务：根据输入的两个银行存款账号A1和A2，以及转账金额 $X=200$ ，从账号A1转 X 元到账号A2，即：账号A1的金额减去 X ，账号A2的金额增加 X 。在应用程序及DBMS中的执行处理过程如下（其中： **$READ(X, t)$** 表示将账号 X 的金额读入内存变量 t ， **$WRITE(X, t)$** 表示将内存变量 t 的值作为账号 X 的金额写入数据库）

应用程序：Process P1

```
update A
set balance = balance - 200.00
where A.aid = 'A1';

.....

update A
set balance = balance + 200.00
where A.aid = 'A2';
```

在DBMS中对DB的访问执行过程

```
→  $READ(A1, t);$ 
    $t := t - 200;$ 
→  $WRITE(A1, t);$ 

→  $READ(A2, t);$ 
    $t := t + 200;$ 
→  $WRITE(A2, t);$ 
```

- ❑ 对该应用程序而言，数据库中数据的一致性就是指：账号A1和账号A2的金额之和不变！

2、定义事务

□ 在现有的数据库管理系统中，事务的定义方式如下：

➤ 事务的启动：隐式方式

- 由数据库管理系统来决定，何时为何用户启动一个新的事务

➤ 事务的结束

- 用户调用commit或rollback命令，显式地结束当前事务
- 由数据库管理系统强行结束一个用户的当前正在运行中的事务

事务结束方式

❑ COMMIT

- 事务正常结束
- 提交事务的所有操作（读+更新）
- 被提交的事务对数据库的所有更新结果被写回到磁盘上的物理数据库中，确保事务执行结果的持久化

❑ ROLLBACK

- 事务异常终止
- 事务运行的过程中发生了故障，不能继续执行
- 数据库管理系统将负责撤销被异常终止的事务对数据库的所有已完成的操作，并将数据库中的数据回退到事务开始时状态

3、有关事务的SQL语句 (1)

❑ ‘事务’除了由一组对于数据库的访问操作构成以外，通常还应该包括少量的事务控制语句，用于定义一个事务的开始和结束(包括正常结束和非正常结束)。因此，与事务有关的控制语句主要有三条：

➤ 事务的开始 (**begin transaction**)

➤ 事务的结束

① 正常结束

- 提交事务 (**commit transaction**)

② 非正常结束

- 回退事务 (**rollback transaction**)

3、有关事务的SQL语句 (2)

□ Begin transaction

- 现有的关系数据库管理系统并没有提供一个用于定义从什么时候开始一个事务的控制语句，事务的启动是隐式的
- 可以通过三种方式来启动一个新的事务：
 - ① 数据定义命令(DDL)
 - ② 将系统设为自动提交方式（打开自动提交标志）
 - ③ 数据操纵命令(DML)

3、有关事务的SQL语句 (3)

□ 事务的启动方式

① 数据定义命令(DDL)

- 每一条数据定义命令都将被作为一个单独的事务来执行
- 在此之前，用户当前正在运行的事务(active事务)将被自动提交

② 将系统设为自动提交方式（打开自动提交标志）

- 每一条数据库访问命令都将被作为一个单独的事务来执行，并根据执行结果由DBMS自动完成提交或回退

③ 数据操纵命令(DML)

- 当DBMS接收到来自某个用户的数据操纵请求时，首先检查该用户当前有没有正在运行的active事务：
- 如果有，该请求被作为当前active事务的组成部分去执行；
- 如果没有，DBMS将自动为该用户启动一个新的事务，该请求就被作为新事务的第一条数据访问请求去执行。

3、有关事务的SQL语句 (4)

□ Commit transaction

- 用于提交用户当前的active事务。如果能够成功完成提交，该事务在执行过程中对于数据库的所有修改操作结果都将永久地反映到数据库中，并且不可被取消。
- 事务的提交操作也可能失败，其原因包括：
 - 发生系统故障
 - 在提交阶段执行的数据完整性检查不满足要求
- 在事务提交失败后，用户可以通过回退(Rollback)操作来取消当前事务
 - 由系统自动提交的事务，如果提交失败，系统将自动执行事务的回退操作

3、有关事务的SQL语句 (5)

□ Rollback transaction

- 取消当前**active**事务在执行过程中的所有操作结果，将数据库回滚至该事务开始前的状态，以便重新执行或放弃（**abort**）该事务
- 事务的执行过程不可能被‘回滚’，执行**rollback**操作是为了撤销该事务在之前的执行过程中对数据库中的数据所作的修改

□ 保存点 (savepoint)

- 在事务的执行过程中，用户可以为该事务设置若干个保存点
- 用户事务可以使用 **Rollback** 命令将当前事务回退到前面的某个保存点**sp**，放弃“在保存点**sp**之后，回退操作之前”执行过的对数据库的所有访问操作，并继续执行当前事务
- 不带保存点的回退操作将结束并放弃整个事务

3、有关事务的SQL语句 (6)

❑ 除了上述三条事务控制语句外，数据库管理系统通常还会提供下述几条与事务有关的控制命令(DCL)

① 设置事务的自动提交命令

SET AUTOCOMMIT ON | OFF ;

② 设置事务的类型

SET TRANSACTION READONLY | READWRITE ;

③ 设置事务的隔离级别

**SET TRANSACTION ISOLATION LEVEL
READUNCOMMITTED |
READCOMMITTED |
READREPEATABLE |
SERIALIZABLE ;**

❑ 用户可以使用这三条命令来设置自己事务的运行方式！

3、有关事务的SQL语句 (7)

❑ SET TRANSACTION READONLY | READWRITE ;

➤ 用于定义，在这之后启动的所有事务在执行过程中对数据库的访问方式

- **READONLY：只读型事务**

- 在事务的运行过程中只能执行对数据库的‘**读**’操作，而不能执行‘**写**’类型的操作
- 直到定义新的事务类型

- **READWRITE：读/写型事务**

- 在事务运行过程中可以执行对数据库的‘**读/写**’操作
- 这是事务的缺省类型定义

3、有关事务的SQL语句 (8)

SET TRANSACTION ISOLATION LEVEL

READUNCOMMITTED |

READCOMMITTED |

READREPEATABLE |

SERIALIZABLE ;

- ❑ 定义当前用户的事务与其它并发执行事务之间的隔离级别；
- ❑ 有四种不同的隔离级别可供选择；
- ❑ 事务的‘隔离级别’与事务并发所采用的封锁策略紧密相关。选择不同的隔离级别，系统所采用的封锁策略则不同！

➤ 基于封锁的并发控制（基本思想）：事务T，数据对象O

- 当事务T持有数据O上的封锁时，允许T以‘读’方式访问 O
- 当事务T持有数据O上的‘写’封锁时，允许T以‘写’方式访问 O
- 一个事务在一个数据对象上，最多只能持有1把一种类型的‘锁’

3、有关事务的SQL语句 (9)

① READUNCOMMITTED: 未提交读

- 在该方式下，当前事务不需要申请任何类型的封锁，因而可能会‘读’到其他并发事务未提交的修改结果
- 禁止一个事务以该方式去执行对数据的‘写’操作，以避免与其它并发事务的‘写’冲突现象

② READCOMMITTED: 提交读

- 在‘读’一个数据对象A之前，需要先申请并获得数据对象A上的‘共享性’封锁，在‘读’操作执行结束之后立即释放该封锁
- 以避免读取到其它并发事务未提交的修改结果

③ READREPEATABLE: 可重复读

- 在‘读’数据对象A之前需要先申请并获得数据对象A上的‘共享性’封锁，并将该封锁维持到当前事务的结束
- 可以避免其它的并发事务对当前事务正在使用的数据对象的修改

④ SERIALIZABLE: 可序列化(可串行化)

- 并发事务以一种可串行化的调度策略实现其并发执行，以避免它们相互之间的干扰现象

3、有关事务的SQL语句 (10)

- 不管采用何种隔离级别，在一个事务以‘写’方式访问数据对象A之前，需要先申请并获得数据对象A上的‘排它性’封锁，并将该封锁维持到当前事务的结束。

	Read 操作		Write 操作	
	锁类型	封锁时间	锁类型	封锁时间
READUNCOMMITTED 未提交读	No Lock	—	不允许执行 Write 操作	
READCOMMITTED 提交读	共享锁	读操作	排它锁	事务
READREPEATABLE 可重复读	共享锁	事务		
SERIALIZABLE 可串行化	共享锁	事务		

事务的隔离级别与封锁策略之间的关系

4、事务的特性（ACID特性）

□ 事务的ACID特性：

➤ 原子性（Atomicity）

- 事务是数据库的逻辑工作单位

- 事务中包括的诸操作要么都做，要么都不做

➤ 一致性（Consistency）

➤ 隔离性（Isolation）

➤ 持续性 / 持久性（Durability）

一致性

- ❑ 事务执行的结果 必须是 使数据库从一个 一致性状态 变到 另一个 一致性状态
- ❑ 一致性状态
 - 数据库中只包含 已成功完成提交的事务 的执行结果
- ❑ 不一致状态
 - 数据库系统运行中发生故障，有些事务尚未完成就被迫中断；
 - 这些未完成事务对数据库所做的修改有一部分已写入物理数据库，这时数据库就处于一种不正确的状态

隔离性、持续性

□ 隔离性

- 一个事务的执行不能被其他事务干扰
- 一个事务内部的操作及使用的数据对其他并发事务是隔离的
- 并发执行的各个事务之间不能互相干扰

□ 持续性也称永久性、持久性 (Permanence)

- 一个事务一旦提交，它对数据库中数据的改变就应该是永久性的。
- 接下来的其他操作或故障不应该对其执行结果有任何影响。

小结

❑ 事务的概念和性质

- **事务**是数据库的逻辑工作单位
- 数据库管理系统保证系统中一切事务的**原子性、一致性、隔离性和持续性**，就保证了事务处于一致状态

❑ **事务** 不仅是恢复的基本单位，也是并发控制的基本单位

❑ 与事务管理有关的SQL命令

- 事务的结束命令：COMMIT, ROLLBACK
- 三种事务设置命令

SET AUTOCOMMIT ON | OFF ;

SET TRANSACTION READONLY | READWRITE ;

SET TRANSACTION ISOLATION LEVEL

READUNCOMMITTED | READCOMMITTED | READREPEATABLE | SERIALIZABLE ;